

OOPs - 2

→ Package

→ Static

→ Singleton Class

→ In-built methods



Packages are containers for classes. They are used to keep the class name space compartmentalized. For example, a package allows you to create a class named List, which you can store in your own package without concern that it will collide with some other class named List stored elsewhere. Packages are stored in a hierarchical manner and are explicitly imported into new class definitions.

The package is both a naming and a visibility control mechanism.

The following statement creates a package called MyPackage: `package MyPackage;`

Java uses file system directories to store packages. For example, the .class files for any classes you declare to be part of MyPackage must be stored in a directory called MyPackage. Remember that case is significant, and the directory name must match the package name exactly.

A package hierarchy must be reflected in the file system of your Java development system. For example, a package declared as

```
package java.awt.image;
```

needs to be stored in `java\awt\image` in a Windows environment. Be sure to choose your package names carefully. You cannot rename a package without renaming the directory in which the classes are stored.

How does the Java run-time system know where to look for packages that you create? The answer has three parts.

- First, by default, the Java run-time system uses the current working directory as its starting point. Thus, if your package is in a subdirectory of the current directory, it will be found.
- Second, you can specify a directory path or paths by setting the CLASSPATH environmental variable.
- Third, you can use the `-classpath` option with `java` and `javac` to specify the path to your classes.

When a package is imported, only those items within the package declared as public will be available to non-subclasses in the importing code.

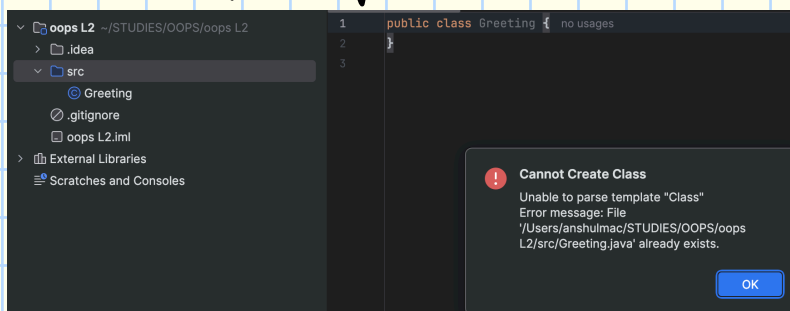
Package

↳ It is container for classes.

↳ They are used to keep the class name compartmentalized.

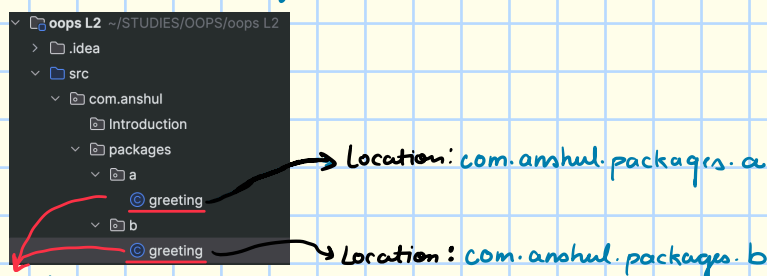
why?

↳ Assume greeting name ki 2 class bnani ha



In Java we cannot do this
↳ like to make more than one
.java file in src

★ Use package put these inside different packages and access those classes through package



both are inside
diff. package.

Import

Using Import statement: Can use other java packages function into another package by importing that package.

```
src
├── com
│   └── anshul
│       ├── Introduction
│       └── packages
│           ├── a
│           │   ├── greeting
│           │   └── message
│           └── b
│               ├── greeting
│               └── message
package com.anshul.packages.a;
import static com.anshul.packages.b.message.message;
public class greeting {
    public static void main(String[] args) {
        System.out.println("Hello World");
        message(); // calling from message function from package b
    }
}
```

In same package
no need to import

Use this in greeting
which belongs to package a

```
package com.anshul.packages.b;
public class message {
    public static void main(String[] args) {
    }
    // Rename usages
    public static void message() {
        System.out.println("this is awesome");
    }
}
```

if it is private, no other package
will be able to use it.

com.anshul.packages.b.message.message;

Path

Java class

method inside
message class

Static element example.

```
staticExample
├── Human
└── main
```

Human.java

package com.anshul.staticExample;

```
public class Human {
    int age;
    String name;
    int salary;
    boolean married;
    static long population;
    public Human(int age, String name, int salary, boolean married) {
        this.age = age;
        this.name = name;
        this.salary = salary;
        this.married = married;
        Human.population += 1;
    }
}
```

main.java

package com.anshul.staticExample;

```
public class Main {
    public static void main(String[] args) {
        Human kunal = new Human(22, "Kunal", 11000, false);
        Human Rahul = new Human(34, "Rahul", 15000, true);
        Human Arpit = new Human(34, "Arpit", 15000, true);

        System.out.println(Human.population); // 2
        System.out.println(Human.population); // 2
        System.out.println(Human.population); // 3
    }
}
```

```
System.out.println(kunal.population);
System.out.println(Rahul.population);
System.out.println(Arpit.population);
```

will work but use class name

When you are accessing, modifying, declaring
static variable don't do it via the
object name, works but still don't use.

Convention → use with class name.

Work with both

→ Convention is to write with
class name, if is static

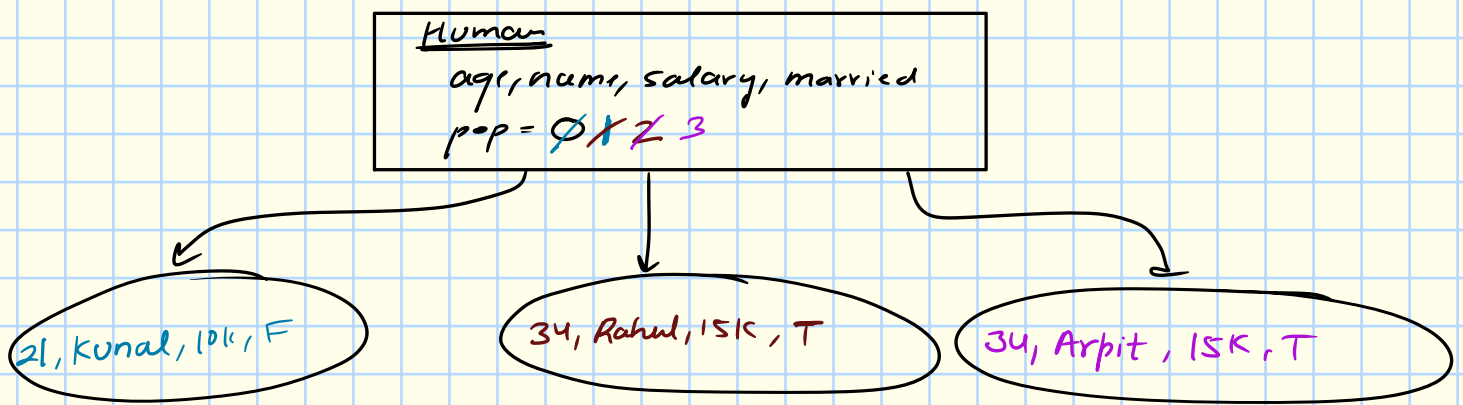
population → is smthg which is common for all
human make it static → becoz same for every object
of type human.

this.population or Human.population

If h1, h2, h3 are 3 human. Human.population

```
h1.population
h2.population
h3.population
```

but this is same for everyone.



What is static?

NOTE: You can use the main method, function without creating an object of that class

⇒ public class main {
 public static void main(String [] args) {

first thing which run whenever you run the java class

→ If not put main static, in order to run anything inside the class we need to create its object.

→ You should be able to run this psvm without creating an object of the main class, that's why it is static.

public class main

Non-static member inside static

ex:-

static function me non-static call nhi kr skte hai

No Instances

```

public class Main {
    psvm(...) {
        greeting();
    }
    // we know that smthng which is not static, belongs to an object
    void greeting() {
        sout("Hello world");
    }
    // Not dependent on object.
    static void fun() {
        greeting(); // Can't use this becoz, it requires an instances.
    }
    // but the func you're using it in does not depend on instances.
}

```

→ This is going to have an instance

Non-static method 'greeting()' cannot be referenced from a static context
Make 'greeting' static More actions...

Static member inside a non-static

ex:-

```

void greeting() {
    sout("Hello world");
    fun(); → static inside N-S
}

```

```

static void fun() {
    Main obj = new Main();
    obj.greeting();
}

```

Can't access non static stuff without referencing their instances in a static context

"this" keyword inside static

represent object

```
ex:- public class A {  
    int age;  
    static void message() {  
        sout(this.age); // can't use non-static inside static  
    }  
    public A(int age) {  
        this.age = age;  
    }  
}
```

Initialization of static variable.

// is demo to show initialization of static variables.

```
ex:- public class StaticBlock {  
    static int a = 4;  
    static int b;  
    static {  
        sout("I'm in static block");  
        b = a * 5;  
    }  
    prvm(... ) {  
        StaticBlock obj = new StaticBlock();  
        sout(StaticBlock.a + "-" + StaticBlock.b);  
    }  
}
```

Output:

I'm in static block

4 20

```
ex:- public class StaticBlock {  
    static int a = 4;  
    static int b;
```

// will only run once, when the first obj is create i.e. when the class is loaded for the first time

```
static {  
    sout("I'm in static block");  
    b = a * 5;  
}
```

```
prvm(... ) {
```

```
    StaticBlock obj = new StaticBlock();
```

```
    sout(StaticBlock.a + "-" + StaticBlock.b);
```

```
    StaticBlock.b += 3;
```

```
    sout(StaticBlock.a + "-" + StaticBlock.b);
```

```
    StaticBlock obj2 = new StaticBlock();
```

```
    sout(StaticBlock.a + "-" + StaticBlock.b);  
}
```

Output

I'm in static block

4 20

4 23

4 23

Inner Classes

Note: • Outside side classes can't be static, only inner classes can be.

⇒ test class inside class

```
package com.anshul.staticExample;

public class InnerClasses {
    class test { 4 usages
        String name; 1 usage
        test(String name) { 2 usages
            this.name = name;
        }
    }

    public static void main(String[] args) {
        test a = new test( name: "Maths");
        test b = new test( name: "English");
    }
}
```

this class is dependent on outer class, and it don't have any object

And will work fine if put it outside

```
com.anshul.staticExample.InnerClasses.this cannot be referenced from a static context
Main test static \0-0- More actions... \0-0-
@ com.anshul.staticExample.InnerClasses.test
@Contract(pure = true) *
test
String name
ops 1.2
```

⇒ test class outside the class

```
package com.anshul.staticExample;

class test { 4 usages
    String name; 1 usage
    test(String name) { 2 usages
        this.name = name;
    }
}

public class InnerClasses {
    public static void main(String[] args) {
        test a = new test( name: "Maths");
        test b = new test( name: "English");
    }
}
```

becoz now this (test) class is not dependent on any other class.

To use diff. classes

↳ non-static knows but sbka scope only vkhoo → not dependent ho.

↳ static knows but use inside main class

→ sout(a.name) // Maths
→ sout(b.name) // English

⇒ test class inside class but it is static

```
package com.anshul.staticExample;

public class InnerClasses {
    static class test { 4 usages
        String name; 1 usage
        test(String name) { 2 usages
            this.name = name;
        }
    }

    public static void main(String[] args) {
        test a = new test( name: "Maths");
        test b = new test( name: "English");
    }
}
```

And made the static, now it's no need to have object

making static means → Now this is actually not dependent on the object of Class (InnerClasses)

→ sout(a.name) → Maths
→ sout(b.name) → English.

```
class test {
    static String name;
    test(String name) {
        test.name = name;
    }
}

public class InnerClasses {
    psvm(...) {
        Test a = new Test("Maths");
        Test b = new Test("English");

        sout(a.name);    // English
        sout(b.name);    // English
    }
}
```

Static-methods resolve during compile-time
As they don't have object of itself.

• dot → It binds the instance variable & instance methods with the reference variables

Internal Working of some statements.

```
public class InnerClasses {  
    static class Test {  
        String name;  
        public Test(String name) {  
            this.name = name;  
        }  
    }  
    public static void main(...) {  
        Test a = new Test("Math");  
        System.out(a); // com.anshul.StaticExample.InnerWorking$Test@9f6046  
    }  
}
```

if we try to print object directly will give this as the internally it happens.

→ but can override this & can return whatever we want.

```
public class InnerClasses {  
    static class Test {  
        String name;  
        public Test(String name) {  
            this.name = name;  
        }  
        @Override  
        public String toString() {  
            return name;  
        }  
    }  
    public static void main(...) {  
        Test a = new Test("Math");  
        System.out(a); // Math  
    }  
}
```

Singleton Class

↳ when only one instance allowed

It is a design pattern that ensures a class has only one instance throughout the application lifecycle & provides a global access point to it.

→ To do this: not allow anyone to use constructor So, no one can create objects.

main.java

```
package com.anshul.Singleton;  
  
public class main {  
    public static void main(String[] args) {  
        Singleton obj1 = Singleton.getInstance();  
  
        Singleton obj2 = Singleton.getInstance();  
  
        Singleton obj3 = Singleton.getInstance();  
    }  
}
```

Singleton.java

```
package com.anshul.Singleton;  
  
public class Singleton { 5 usages  
    // private constructor so no one can access other than this class  
    private Singleton() { 1 usage  
        System.out.println("Hello this is Singleton class");  
    }  
  
    private static Singleton instance; // instance initialized of type Singleton  
  
    public static Singleton getInstance() { 1 usage  
        if(instance == null){  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

method → with return type of singleton

